

Aegis Brain: Technical Documentation

1. Overview

Aegis Brain is an AI-powered Industrial Knowledge Intelligence platform designed to solve the "Knowledge Cliff" and disconnected data silos that plague asset-intensive industries. It serves as an **Expert Knowledge Copilot**, acting as a Unified Asset & Operations Brain.

2. System Workflows

2.1 Universal Document Ingestion Pipeline

When a user uploads a document (e.g., PDF, TXT) via the drag-and-drop interface:

- File Reading & Base64 Encoding:** The frontend reads the file in binary format and converts it to Base64 to safely transmit it over HTTP via JSON.
- Text Extraction (OCR & Document Intelligence):** The Python backend receives the payload, decodes it, and passes it to the `pypdf` extraction engine. The engine reads the binary stream and extracts all raw text content from the embedded pages.
- Knowledge Graph Vectorization:** The extracted text is added to the in-memory `DOCUMENTS` array, acting as a real-time Document Store.

2.2 Retrieval-Augmented Generation (RAG) Engine

When a technician asks a query (e.g., "What is the RCA for recent pump failures?"):

- Mathematical Term Frequency-Inverse Document Frequency (TF-IDF):** The query and all documents are tokenized, ignoring common stop words. The system calculates the Term Frequency (TF) and smoothed Inverse Document Frequency (IDF) to create mathematical vectors representing semantic weight.
- Cosine Similarity Scoring:** The engine computes the dot product of the query vector and the document vectors to identify the document with the highest Cosine Similarity score.
- Sentence-Level Granularity:** Instead of returning a massive document block, the engine breaks the winning document into individual sentences. It runs the Cosine Similarity calculation again against each sentence to find the exact phrase that answers the query.

2.3 Agentic AI & Copilot Generation

Once the best semantic match is found, the Expert Knowledge Copilot acts on the data:

1. **Confidence Scoring:** The raw mathematical similarity score is converted into a percentage. This dynamically triggers color-coded UX badges (Green > 80%, Orange > 40%, Red < 40%), ensuring technicians don't trust hallucinations.
2. **Knowledge Graph Ontology Extraction:** The engine analyzes the target sentence for proper nouns and industrial terminology (capitalized words over 3 characters) and dynamically tags them as "Ontology Nodes" (e.g., `Pump` , `Hazard`).
3. **Response Synthesis:** The UI formats a premium, clear response containing the synthesis, exact citation excerpt, and a direct link to the originating PDF.
4. **Agentic Workflows:** The UI offers proactive next steps (e.g., "Auto-Generate QMS Compliance Report"), simulating cross-functional AI automation.

3. Architecture Highlights

- **Zero-Dependency Lightweight Backend:** Uses Python's native `http.server` combined with mathematical vectorization written from scratch. This guarantees 100% data privacy since no data is sent to external APIs.
- **Responsive Web UI:** Built with Vanilla JS and CSS for ultimate speed and compatibility on mobile devices out in the field.